Beam Design with ANSYS and AI Agents — Part 1

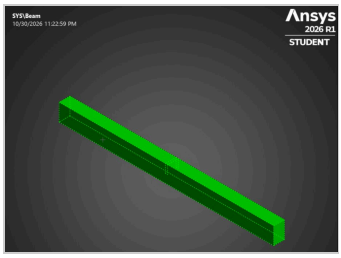
Team Logistics · Purdue VIP Agentic AI · Repository: github.com/sais03687/team-logistics-beam-agent · Sai Suram, Alex, Jaxson

1. Problem and approach

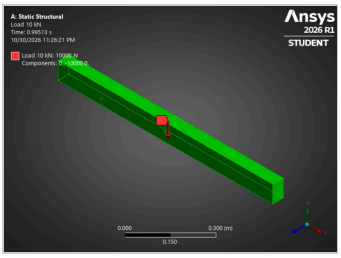
A simply supported rectangular steel beam (L = 1000 mm, central downward load P = 10 kN, no self-weight, E = 200 GPa, ν = 0.30, ρ = 7850 kg/m³) is sized for minimum mass with 20 ≤ b ≤ 100 mm and 40 ≤ h ≤ 120 mm, subject to midspan deflection ≤ 1.0 mm and nominal midspan bottom-surface von Mises stress ≤ 120 MPa. Two evaluation tools are available: the provided analytical model (`beam_model.py`, Euler–Bernoulli, unchanged) and a parameterized 3D solid ANSYS Mechanical model driven from Python. An OpenAI model (`gpt-4.1`) proposes designs and chooses the tool; a Python controller validates every proposal and enforces permissions, bounds and budgets.

2. Task 1 — ANSYS model and verification

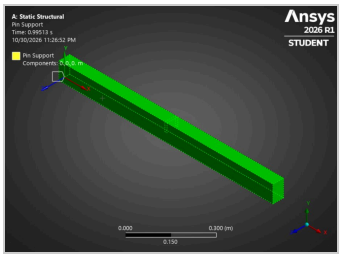
Model. Geometry is built by a SpaceClaim script (`beam_geometry_spaceclaim.py`; X = span, Y = height h, Z = width b, origin at the left bottom edge). A 10 mm wide strip is imprinted on the top face at midspan and receives the 10 kN load (−Y). Supports are line supports on the bottom edges: a **pin** at x = 0 ($u_x = u_y = u_z = 0$) and a **roller** at x = L ($u_y = 0$, x and z free). Fixing z on the pin edge only removes the rigid-body mode; it does not restrain bending. Faces and edges are found by bounding box and saved as named selections (NS_LOAD_STRIP, NS_PIN_EDGE, NS_ROLLER_EDGE), so the setup survives every geometry change. The mesh is a hexahedral sweep with an 8 mm global size and 5 mm on the strip (search mesh). **Nominal stress** is the average von Mises stress of the bottom-surface nodes with 495 ≤ x ≤ 505 mm and $|z| \le 5$ mm (worksheet named selection NS_NOMINAL_PROBE). The global peak stress is not used: it sits at the support edges and the strip corners, where line supports and the step in load create singularities.



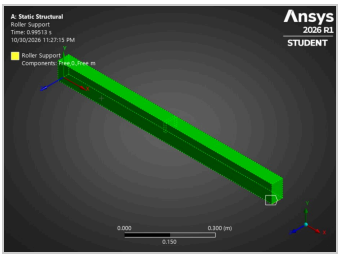
(a) Solid with 10 mm strip, 50×80 mm



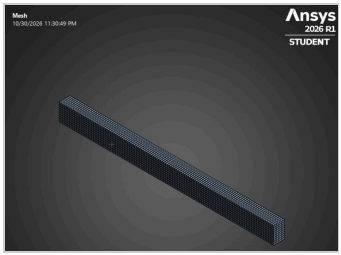
(b) 10 kN in −Y on NS_LOAD_STRIP



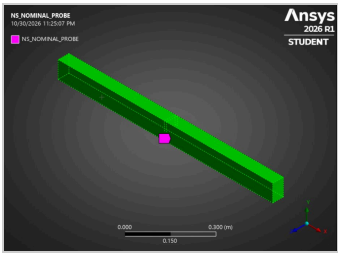
(c) Pin: x, y, z = 0 on left bottom edge



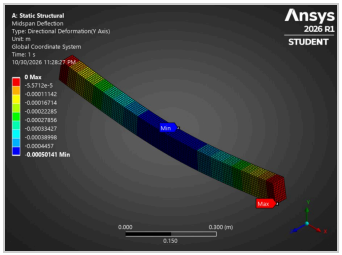
(d) Roller: y = 0 on right bottom edge



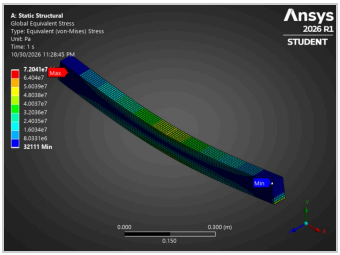
(e) Search mesh: 41 870 nodes, 8 750 elements



(f) NS_NOMINAL_PROBE: 8 bottom nodes at midspan



(g) u_y: −0.501 mm at midspan



(h) Global von Mises: 72 MPa peak at the support (singular)

Mesh refinement and equilibrium (start design 50 × 80 mm)

Global / strip size (mm)	Nodes	Elements	Deflection (mm)	Nominal stress (MPa)	Global peak (MPa)	$R_{pin} + R_{roller}$ (N)
12 / 5	15 260	2 975	0.50102	46.066	49.0	10 000.0
8 / 5	41 870	8 750	0.50141	46.139	72.0	10 000.0
6 / 5	106 347	23 380	0.50176	46.163	98.9	10 000.0

Between the two finest meshes the deflection changes by **0.07 %** (limit 2 %) and the nominal stress by **0.05 %** (limit 5 %); reactions balance the load to **0.00 %** (limit 1 %; each support carries 5000 N). The global peak keeps rising with refinement (49 → 72 → 99 MPa), which confirms it is a singularity and justifies the nominal probe. The 8 mm mesh is used for all search evaluations; the final designs are re-checked on three meshes in Section 5.

Python vs ANSYS at the five required designs

b × h (mm)	Mass (kg)	δ Python	δ ANSYS	Signed Δδ (mm)	|Δδ| %	σ Python	σ ANSYS	Signed Δσ (MPa)	|Δσ| %	Feasible Py / AN
50 × 80	31.40	0.4883	0.5014	−0.0131	2.62	46.88	46.14	+0.74	1.59	yes / yes
20 × 40	6.28	9.7656	9.8179	−0.0523	0.53	468.75	463.87	+4.88	1.05	no / no
20 × 120	18.84	0.3617	0.3892	−0.0276	7.08	52.08	50.94	+1.14	2.25	yes / yes
100 × 40	31.40	1.9531	1.9678	−0.0147	0.75	93.75	91.90	+1.85	2.02	no / no
100 × 120	94.20	0.0723	0.0780	−0.0057	7.29	10.42	10.17	+0.25	2.45	yes / yes

δ in mm, σ = nominal stress in MPa; signed difference = Python − ANSYS, percentage relative to ANSYS. Masses agree exactly.

Interpretation. Python always under-predicts deflection, and the gap grows with depth: 0.5–0.8 % at h = 40 mm but about 7 % at h = 120 mm. The cause is **shear deformation**, which Euler–Bernoulli theory omits; its share of the midspan deflection scales with $(h/L)^2$ (Timoshenko: $\delta_s/\delta_b = 2(1+\nu)/\kappa\cdot(h/L)^2 \approx 3.1\cdot(h/L)^2$, i.e. 2.0 % at h = 80 mm and 4.5 % at h = 120 mm). Local indentation under the strip and at the line supports adds the rest. Python over-predicts the nominal stress by 1–2.5 %: the 10 mm strip spreads the load (M = PL/4 − Ps/8, −0.5 %) and the probe averages a small region where 3D effects slightly relax the bottom-fibre stress. Both tools agree on feasibility at all five points, but near the deflection limit Python is **unconservative by up to 7 %**, so a Python-only optimum can violate the constraint in ANSYS.

3. Task 2 — Python integration

`evaluate_python(b, h)` (`beam_tools.py`) and `evaluate_ansys(b, h)` (`ansys_tools.py`) return the same dictionary: width_mm, height_mm, model_source, mass_kg, deflection_mm, nominal_stress_mpa, status, feasible, deflection_margin_mm, stress_margin_mpa, elapsed_s, cache_hit, evaluation_id, ansys_mesh (sizes, nodes, elements), project_version, error. `evaluate_ansys` keeps one **persistent Workbench session** (PyWorkbench, started once; startup 57–73 s, reported separately), rebuilds the geometry through a SpaceClaim journal, and meshes, solves and extracts results through PyMechanical. Mechanical restarts after each geometry update, so the client reconnects automatically. One evaluation takes 200–250 s (Python: < 1 ms).

- **No stale or mislabelled results.** A result is accepted only if the solve status is Done, the solved volume equals b·h·L within 0.1 % (proves the new geometry was solved) and the reactions balance the load within 1 %. Otherwise status = "error", all numbers are None and nothing is cached. There is no Python fallback inside `evaluate_ansys`; the controller also checks that the returned model_source matches the requested tool.
- **Cache key** = tool, b, h, model version, mesh sizes, load case, material.
- **Geometry-update robustness.** Rebuilding the body breaks Mechanical's material link (a blocking "Invalid Assignment" error), so the solve script deletes the stale material-assignment object and assigns Structural Steel before each solve; the probe selection is regenerated on the new mesh.

Smoke test (successive geometry changes)	Mass (kg)	δ ANSYS (mm)	σ ANSYS (MPa)	Nodes	elapsed_s	cache_hit
50 × 80	31.40	0.5014	46.14	41 870	249	False
60 × 100 (new geometry, different result)	47.10	0.2183	24.53	60 480	214	False
50 × 80 again	31.40	0.5014	46.14	41 870	0.00001	True

Failure test (`ansys_failure_test.py`, all checks passed): an out-of-bounds design and a simulated ANSYS crash both return status "error" with no numbers and model_source "ansys"; the failed design is not cached, so

a retry calls ANSYS again; a cached success is never served for a different design.

4. Task 3 — Agent policy and controller

One model and one system prompt are used in every workflow. The prompt states the problem, the two tools (with the note that ANSYS includes shear and usually gives a larger deflection), simple scaling facts (mass $\propto bh$, $\delta \propto 1/(bh^3)$, $\sigma \propto 1/(bh^2)$) and the rule that a rejected proposal still costs an iteration. Each user message gives the workflow's allowed tools, the remaining ANSYS budget, the iteration number and the full evaluation history. The model must reply with JSON `{width_mm, height_mm, tool, reason}`. The controller (`beam_agent.py`) parses the JSON, checks the keys, numeric types, bounds, tool permission and ANSYS budget, and only then runs the tools. Invalid proposals are logged as rows with the rejection reason and are shown to the model in the next message. "both" runs Python and ANSYS on the same design and counts as one ANSYS request. After 10 candidate iterations the model recommends a final design, which is always checked with the **reserved** ANSYS evaluation. No numerical optimizer generates candidates; the controller never changes a proposal.

Workflow	Allowed tools in search	ANSYS search budget	Reserved final ANSYS check
A — ANSYS throughout	ansys	10	1
B — Python search + one ANSYS check	python	0	1
C — Adaptive	python, ansys, both	5	1

5. Task 4 — Fair comparison of the three workflows

All three workflows used the same model (`gpt-4.1`), the same system prompt, 10 candidate iterations, the 8 mm search mesh and an empty cache at the start. Every tool execution is one row of `team_logistics_beam_results.csv` (35 rows; no invalid proposals occurred). ANSYS startup (57–63 s) is excluded from the times.

Workflow	Final b × h (mm)	Mass (kg)	ANSYS δ (mm)	ANSYS σ (MPa)	ANSYS-feasible	ANSYS requests / new solves	Python runs	Tool time (min)	Mass vs best verified
A — ANSYS throughout	20 × 86.5	13.581	0.9967	98.6	**yes**	11 / 9	0	34.9	—
B — Python + 1 ANSYS check	20 × 86.0	13.502	1.0138	99.7	**no (+1.4 %)**	1 / 1	10	3.9	not usable
C — Adaptive	21 × 85.0	14.012	0.9991	97.3	**yes**	6 / 5	7	17.2	+3.2 %

"Requests" counts cache hits (A repeated 86.5 mm in iteration 10 and as its final design; C's final was already solved in iteration 9). The ANSYS feasibility limit at b = 20 mm lies between h = 86.2 mm (1.0069 mm) and 86.5 mm (0.9967 mm), i.e. h ≈ 86.4 mm and about 13.56 kg; the Python limit is h = 85.5 mm.

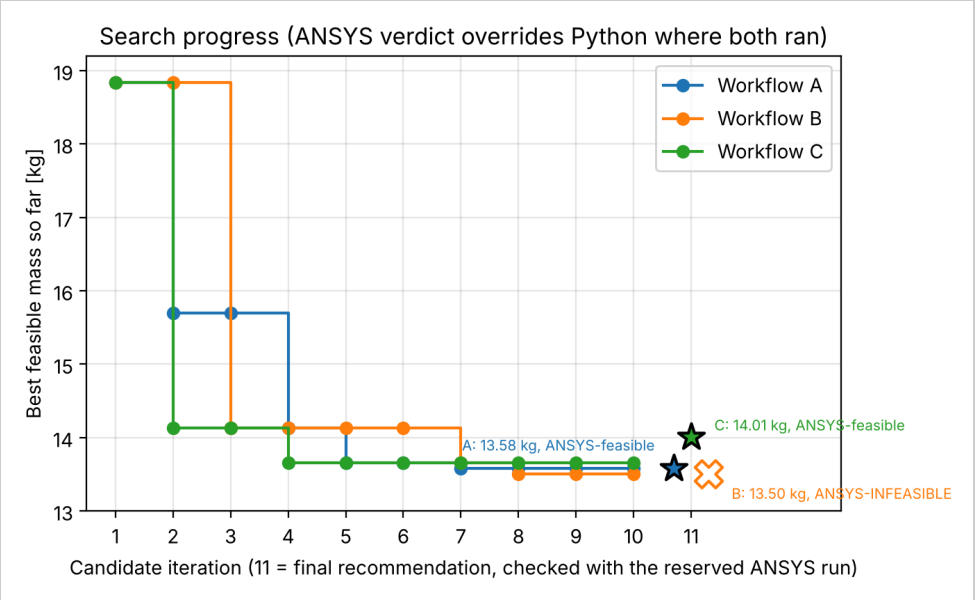


Fig. 2 — Best feasible mass per iteration (ANSYS verdict overrides Python where both ran); markers at 11 are the final ANSYS checks.

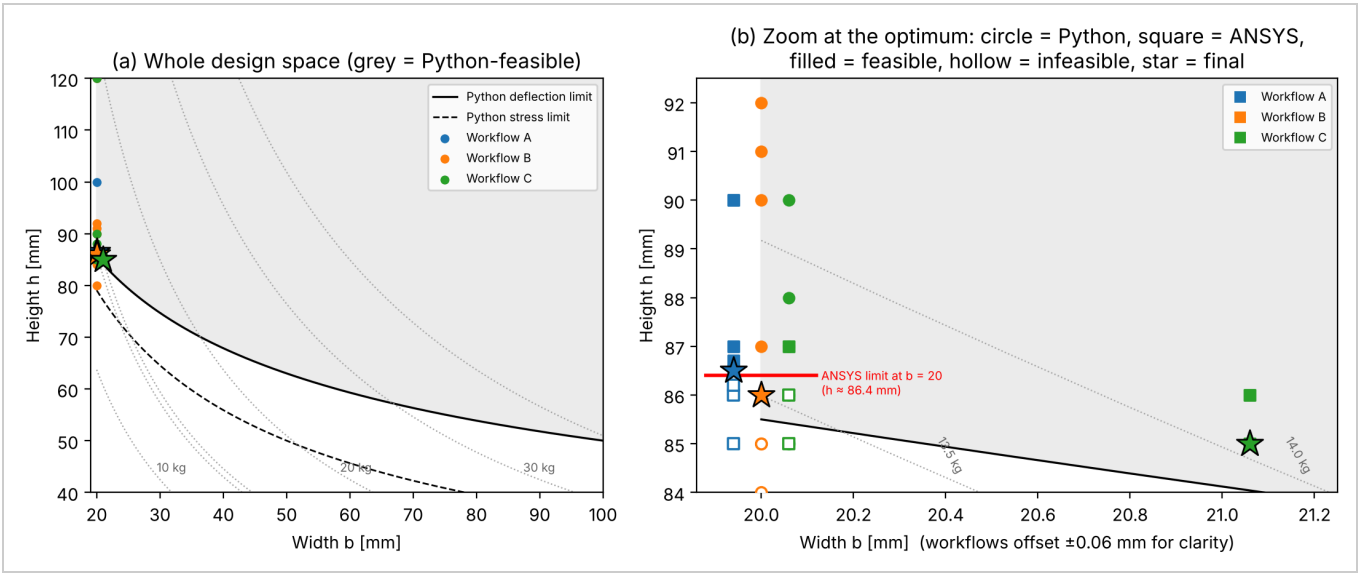


Fig. 3 — Proposals in the design space (a) and zoom at the optimum (b): circle = Python, square = ANSYS, hollow = infeasible, star = final.

Observations. All three agents quickly moved to the narrowest width (b = 20 mm, as the scaling facts suggest) and bisected on h. **B** converged on the Python boundary (h = 86 mm, 0.983 mm in Python) and recommended it; the one ANSYS check showed 1.014 mm, so B's answer violates the deflection limit — the 2.6–3 % shear gap from Task 1 is larger than B's Python margin. **A** bisected in ANSYS down to a 0.3 mm interval and delivered the best verified design (13.58 kg, 0.1 % above the interpolated ANSYS optimum), but needed 9 solves (35 min) and wasted iteration 10 on a repeat. **C** used Python to reach the boundary region cheaply (iterations 1–3), spent ANSYS where the tools disagree (85, 86, 87 mm), and found the ANSYS limit between 86 and 87 mm with 5 solves (17 min). Its weakness was the final choice: it recommended 21 × 85 mm (14.01 kg, margin 0.001 mm) although it had already verified the lighter 20 × 87 mm (13.66 kg) in iteration 4, and it spent iterations 8–9 on b = 21 mm instead of h = 86.5 mm.

Mesh check of the final designs

Final design	δ 10 / 8 / 6 mm mesh (mm)	σ 10 / 8 / 6 mm mesh (MPa)	Nodes (6 mm)	Δδ / Δσ, two finest	Feasible on finest
A: 20 × 86.5	0.9962 / 0.9967 / 0.9976	98.51 / 98.58 / 98.63	50 152	0.08 % / 0.05 %	**yes (margin 0.0024 mm)**

B: 20 × 86.0	1.0132 / 1.0138 / 1.0146	99.67 / 99.74 / 99.79	50 152	0.08 % / 0.05 %	**no (all meshes)**
C: 21 × 85.0	0.9986 / 0.9991 / 0.9999	97.20 / 97.26 / 97.31	50 152	0.08 % / 0.06 %	**yes (margin 0.0001 mm)**

All three final designs are mesh-converged (limits 2 % and 5 %), so the search mesh does not change any verdict. Deflection rises slightly with refinement (about +0.08 % per step), so the 8 mm search mesh is marginally unconservative: C's margin shrinks from 0.0009 to 0.0001 mm on the 6 mm mesh, which is effectively zero; A keeps a small but real margin. B is infeasible on every mesh.

Was adaptive selection worth it?

Yes, compared with B; partly, compared with A. Against B, the extra ANSYS runs were clearly worth it: B's cheap answer is infeasible, and a Python-only search cannot see that its 1.7 % margin is smaller than the model error. Against A, C reached a verified-feasible design with **half the ANSYS time** (17 vs 35 min, 5 vs 9 solves). Its delivered mass was 3.2 % higher than A's, but the information to do better was already in its history: choosing its own best verified design (20 × 87 mm) would have cost only 0.6 % more mass than A. So adaptive selection paid off in the search; the loss came from the agent's final-selection reasoning, not from the tool policy. Two cheap controller improvements would fix this without letting the controller choose designs: show the agent a list of ANSYS-verified designs sorted by mass before the final step, and reject exact repeats (A and B each wasted one iteration). With more agent runs (we ran each workflow once, so LLM randomness is not averaged out), we would expect C to approach A's mass at about half the cost.

6. Reproducibility and limitations

The repository holds all code, the unchanged `beam_model.py`, the Workbench archive `team_logistics_beam.wbpz`, the SpaceClaim and Mechanical scripts, screenshots, logs, all result CSV files and `team_logistics_beam_agent.ipynb`, which rebuilds every table and plot from the CSV files (ANSYS and agent cells run only when enabled). `README.md` gives the run order and setup; no keys, `.env` files or virtual environments are committed. Limitations: one run per workflow (no statistics over LLM randomness); line supports and the load step create local singularities (handled with the nominal probe); the student licence caps meshes near 128k nodes, so the 6 mm mesh is the finest affordable for wide sections. For a design to be built, a small deflection margin (for example 1 %) should be required instead of the bare limit, since C's design passes by only 0.0001 mm.

7. Individual contributions

Member	Contribution
Sai Suram	Code: Python–ANSYS integration (`evaluate_python`, `evaluate_ansys`, cache, failure handling), AI agent and controller, workflow runs, notebook, repository
Alex	ANSYS models: geometry, supports, load, mesh, named selections, nominal stress probe, mesh study, model screenshots
Jaxson	Graphs: comparison plots and result figures